



المدرسة الوطنية للعلوم التطبيقية - بني ملال
ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵏⴰⵢⵜ ⵜⴰⵖⴻⵔⴰⵏⵜ ⵜⴰⵙⴰⵢⵜ
Ecole Nationale des Sciences Appliquées - Béni Mellal

Module :

Génie Logiciel

**Compte Rendu du TP 4 :
Notion & Git**

2^{ème} année : Filière Transformation Digitale Industrielle

Réalisé par :

ECH-CHOKHMANY Loubna

Professeur :

- Pr. BE.ELBAGHAZAOUI

Année universitaire 2025 / 2026

I. Introduction :

Ce rapport présente les travaux pratiques réalisés dans le cadre de l'intégration de Notion et Git pour la gestion de projet. L'objectif principal est de maîtriser l'utilisation combinée de ces deux outils essentiels dans un workflow de développement professionnel :

- Configurer un espace de travail collaboratif sur Notion avec un tableau Kanban.
- Initialiser un dépôt Git et le connecter à GitHub avec plusieurs branches.
- Synchroniser les tâches Notion avec les branches Git en temps réel.
- Documenter les problèmes rencontrés et leurs solutions dans Notion.
- Rédiger un compte-rendu de réunion récapitulatif.

Le projet consiste en une application To-Do List avec trois fonctionnalités principales : ajouter une tâche, supprimer une tâche et marquer une tâche comme terminée. Ces fonctionnalités sont gérées à la fois dans Notion (suivi des tâches) et dans Git/GitHub (gestion du code source).

II. Étape 1 - Mise en place de Notion :

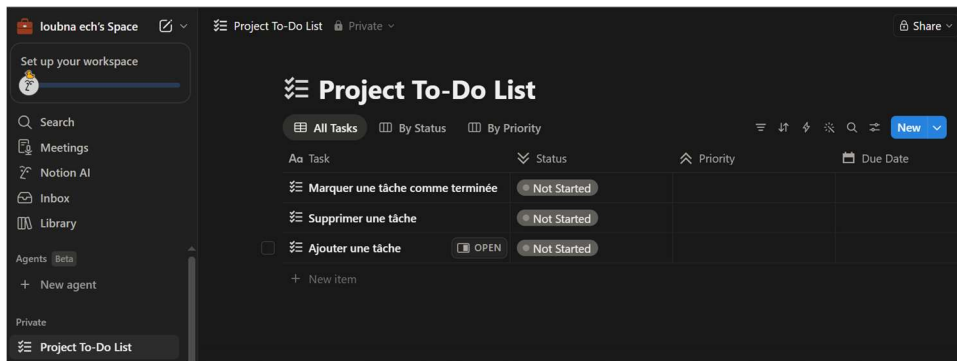
L'objectif de cette étape est de configurer un espace de travail collaboratif sur la plateforme Notion pour organiser et suivre les tâches du projet To-Do List.

1. Création de l'espace de travail :

La configuration débute par la création d'une page dédiée au projet avec les éléments suivants :

- **Compte Notion** : Création d'un compte gratuit sur notion.so permettant d'accéder à toutes les fonctionnalités de gestion de projet collaboratif.
- **Page principale** : Création d'une page intitulée « Project To-Do List » qui sert de hub central pour toute la documentation et le suivi du projet.
- **Vue All Tasks** : Configuration d'un tableau de bord affichant toutes les tâches avec leurs statuts, priorités et dates d'échéance.

Vue All Tasks - Les 3 fonctionnalités du projet ajoutées

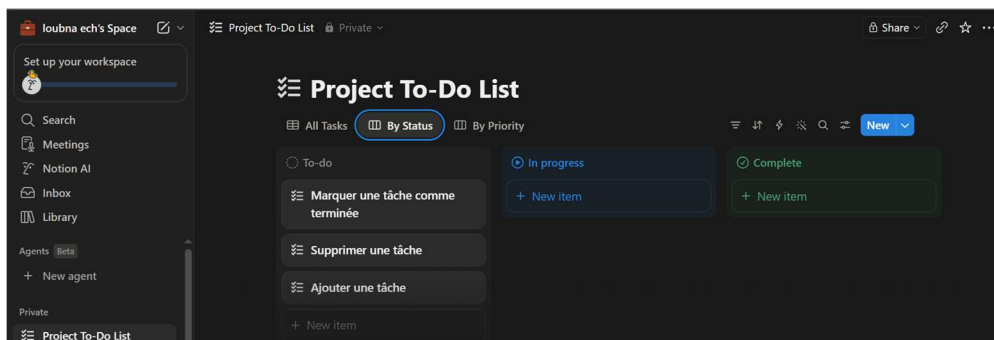


2. Configuration du tableau Kanban :

Le tableau Kanban est l'outil central de suivi du projet. Il permet de visualiser l'avancement des tâches selon trois colonnes distinctes :

- **Colonne « To-do »** : Contient les tâches planifiées mais non encore démarrées. Au départ, les 3 fonctionnalités (Ajouter, Supprimer, Marquer comme terminée) se trouvent dans cette colonne.
- **Colonne « In progress »** : Regroupe les tâches en cours de développement actif, correspondant aux branches Git actives.
- **Colonne « Complete »** : Recense les fonctionnalités entièrement développées, testées et fusionnées dans la branche develop.

Tableau Kanban (vue By Status) - 3 tâches initialement dans la colonne To-do



III. Étape 2 - Initialisation du projet avec Git :

Cette étape consiste à créer et configurer le dépôt Git local, puis à le connecter au dépôt distant GitHub, en mettant en place la structure de branches nécessaire au projet.

1. Création du dépôt local :

La mise en place du projet Node.js et du dépôt Git s'effectue avec les commandes suivantes :

- **Dossier du projet (mkdir todo-list-app)** : Création du répertoire racine destiné à accueillir l'ensemble des ressources du projet.
- **Initialisation Git (git init -b main)** : Transforme le répertoire en dépôt Git avec la branche principale nommée main, conformément aux conventions modernes de GitHub.
- **Fichier README.md** : Création d'un fichier de documentation décrivant le projet, ses objectifs et ses instructions d'installation.
- **Premier commit** : Validation des fichiers initiaux dans l'historique Git, marquant le début officiel du cycle de vie du logiciel.

Initialisation du dépôt Git local et premier commit sur la branche main

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~
$ mkdir todo-list-app

mon pc@DESKTOP-7SSGS3L MINGW64 ~
$ cd todo-list-app

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app
$ git init -b main
Initialized empty Git repository in C:/Users/mon pc/todo-list-app/.git/

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (main)
$ echo "# Application To-Do List" > README.md

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (main)
$ git add README.md
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (main)
$ git commit -m "Initialisation du projet avec README"
[main (root-commit) 0fbdbf] Initialisation du projet avec README
1 file changed, 1 insertion(+)
create mode 100644 README.md

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (main)
$ |
```

2. Configuration des branches et connexion à GitHub :

La structure de branches adoptée suit les bonnes pratiques du développement professionnel avec Git Flow :

- **Branche main** : Branche principale de production, toujours stable et déployable.
- **Branche develop (git checkout -b develop)** : Branche d'intégration où les fonctionnalités sont fusionnées avant d'aller en production.
- **Connexion GitHub (git remote add origin)** : Association du dépôt local au dépôt distant GitHub pour permettre la collaboration et la sauvegarde en ligne.
- **Push initial (git push -u origin main)** : Envoi de la branche main vers GitHub et configuration du tracking pour les futurs push/pull.

Création de la branche develop et push vers GitHub - Connexion établie avec succès

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (main)
$ git checkout -b develop
Switched to a new branch 'develop'

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (develop)
$ git remote add origin https://github.com/Loubna-ECH-CHOKHMANY/todo-list-app.git

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (develop)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 259 bytes | 259.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Loubna-ECH-CHOKHMANY/todo-list-app.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (develop)
$ git push origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote:   https://github.com/Loubna-ECH-CHOKHMANY/todo-list-app/pull/new/develop
remote:
To https://github.com/Loubna-ECH-CHOKHMANY/todo-list-app.git
 * [new branch]      develop -> develop
```

Lien du dépôt GitHub :

<https://github.com/Loubna-ECH-CHOKHMANY/todo-list-app>

IV. Étape 3 - Suivi des tâches et synchronisation Notion / Git :

Cette étape constitue le cœur du TP : la synchronisation entre le tableau Kanban de Notion et les branches Git. Chaque fonctionnalité correspond à la fois à une carte Notion et à une branche Git dédiée.

1. Principe de synchronisation :

La méthodologie adoptée repose sur une correspondance directe entre les outils :

- **Carte Notion → Branche Git :** Chaque fonctionnalité dispose d'une carte dans Notion ET d'une branche Git au format feature/nom-fonctionnalité.
- **Déplacement de carte :** Lorsque le développement d'une fonctionnalité commence, la carte est déplacée de « To-do » vers « In progress » dans Notion.
- **Fermeture de branche :** Une fois la fonctionnalité terminée et fusionnée dans develop, la carte est déplacée vers « Complete ».

2. Création des branches de fonctionnalités :

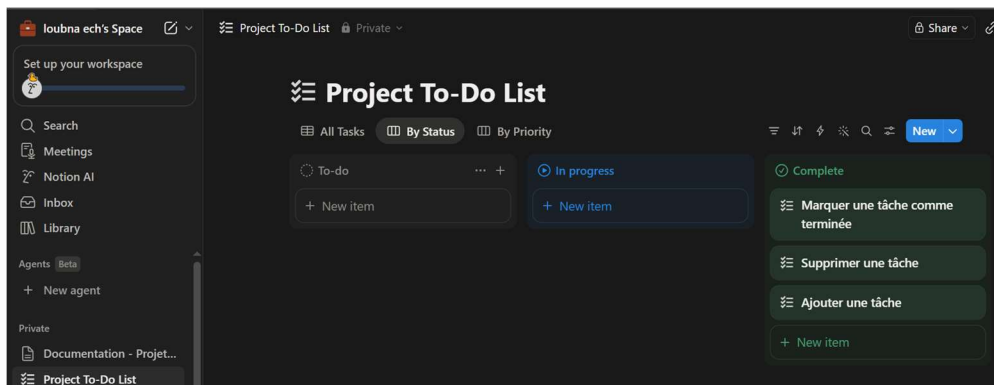
Trois branches ont été créées, chacune correspondant à une fonctionnalité du projet :

- **feature/add-task** : Branche dédiée au développement de la fonctionnalité « Ajouter une tâche ». Contient le fichier add-task.js.
- **feature/delete-task** : Branche dédiée au développement de la fonctionnalité « Supprimer une tâche ». Contient le fichier delete-task.js.
- **feature/complete-task** : Branche dédiée au développement de la fonctionnalité « Marquer une tâche comme terminée ». Contient le fichier complete-task.js.

3. Mise à jour du tableau Kanban :

Une fois toutes les fonctionnalités développées et fusionnées dans develop, les cartes Notion sont déplacées vers la colonne « Complete », reflétant l'état réel du projet dans Git.

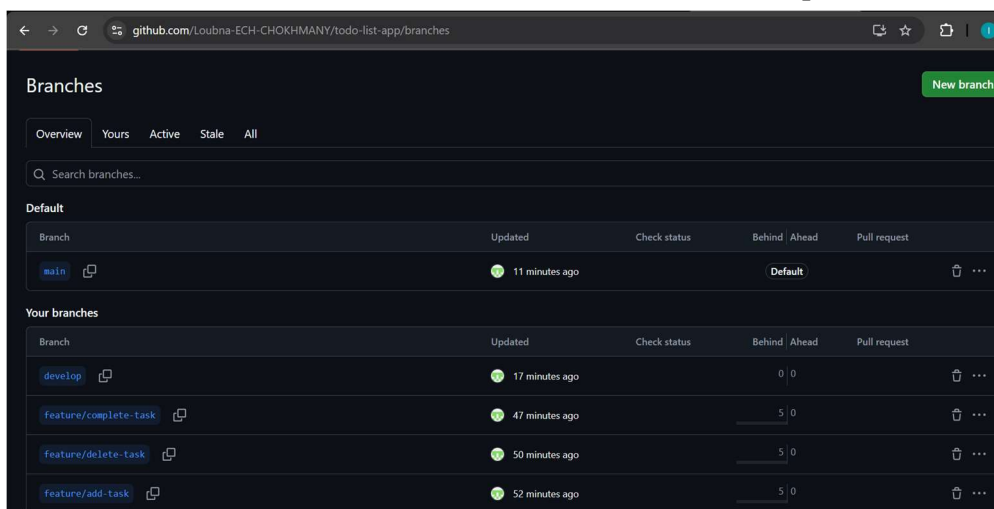
Tableau Kanban mis à jour - Les 3 fonctionnalités dans la colonne Complete



4. Branches GitHub - Vue d'ensemble :

La capture ci-dessous illustre les 5 branches présentes sur GitHub : main, develop, feature/add-task, feature/delete-task et feature/complete-task.

Vue des branches GitHub — 5 branches créées et actives sur le dépôt distant



5. Fichiers du projet sur GitHub :

Le dépôt GitHub contient l'intégralité des fichiers du projet, poussés depuis la branche main après fusion avec develop.

Dépôt GitHub - Les 5 fichiers du projet visibles sur la branche main

 Loubna-ECH-CHOKHMANY	Ajout du code complet de l'application To-Do List	52bd937 · 16 minutes ago
 README.md	Initialisation du projet avec README	
 add-task.js	Ajout du code complet de l'application To-Do List	
 complete-task.js	Ajout du code complet de l'application To-Do List	
 delete-task.js	Ajout du code complet de l'application To-Do List	
 index.js	Ajout du code complet de l'application To-Do List	

V. Étape 4 - Documentation des problèmes dans Notion :

L'objectif de cette étape est de documenter de manière structurée les problèmes rencontrés pendant le développement ainsi que leurs solutions, directement dans la page Notion dédiée à la documentation du projet.

1. Importance de la documentation des problèmes :

La documentation des incidents est une pratique essentielle dans tout projet de développement professionnel. Elle permet de :

- **Capitaliser sur l'expérience :** Chaque problème résolu constitue un apprentissage qui peut bénéficier à l'ensemble de l'équipe lors de situations similaires futures.
- **Accélérer la résolution :** En consultant la base de connaissances, un développeur peut retrouver rapidement la solution à un problème déjà rencontré.
- **Assurer la traçabilité :** Le lien entre un problème et le commit Git correspondant permet de comprendre les raisons de chaque modification du code.

2. Problèmes documentés :

Deux problèmes majeurs ont été rencontrés et documentés dans Notion :

- **Problème 1 - Conflit de fusion Git :** Lors de la tentative de fusion de la branche feature dans main, un conflit de contenu est apparu car les deux branches avaient modifié les mêmes lignes du fichier index.js. La résolution a nécessité une intervention manuelle consistant à combiner les deux versions du fichier en gardant les commentaires de main et la fonction soustraction de feature.

- **Problème 2 - Erreur d'authentification GitHub** : Lors du premier git push vers GitHub, une fenêtre d'authentification s'est ouverte. La solution a consisté à utiliser l'option « Sign in with browser » via le Git Credential Manager, qui gère automatiquement les tokens d'authentification sécurisés.

Page Documentation - Exigences fonctionnelles et section Problèmes rencontrés avec solutions



VI. Étape 5 - Collaboration et feedback :

Cette dernière étape vise à documenter dans Notion une réunion récapitulative du projet, permettant de faire le bilan des travaux réalisés et d'identifier les axes d'amélioration pour les prochaines itérations.

1. Utilité de la réunion récapitulative :

Dans un contexte de développement agile, la réunion de rétrospective (ou sprint review) est une pratique clé. Elle permet à l'équipe de :

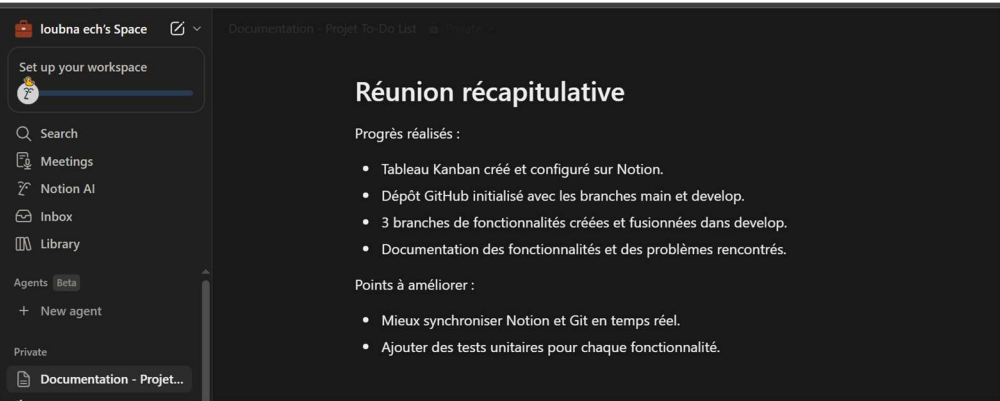
- **Mesurer l'avancement** : Comparer les objectifs initiaux avec ce qui a été effectivement réalisé, en s'appuyant sur l'état du tableau Kanban.
- **Partager les apprentissages** : Discuter des difficultés rencontrées et des solutions trouvées, pour capitaliser sur l'expérience collective.
- **Planifier les améliorations** : Identifier les points faibles du workflow actuel et définir des actions concrètes pour les prochains sprints.

2. Contenu de la réunion :

La réunion récapitulative documentée dans Notion couvre deux volets essentiels :

- **Progrès réalisés :** Tableau Kanban créé et configuré avec les 3 fonctionnalités. Dépôt GitHub initialisé avec les branches main et develop. 3 branches de fonctionnalités créées, développées et fusionnées dans develop. Documentation complète des fonctionnalités et des problèmes rencontrés.
- **Points à améliorer :** Mieux synchroniser Notion et Git en temps réel lors du développement. Ajouter des tests unitaires pour chaque fonctionnalité. Mettre en place des pull requests et des reviews de code sur GitHub avant les merges.

Réunion récapitulative dans Notion - Progrès réalisés et points à améliorer



VII. Code source - Application To-Do List :

L'application To-Do List est développée en Node.js et structurée en modules séparés, chacun correspondant à une fonctionnalité et à une branche Git dédiée.

1. Architecture du projet :

Le projet suit une architecture modulaire avec une séparation claire des responsabilités :

Fichier	Rôle et description
add-task.js	Gère la liste des tâches en mémoire. Fournit la fonction ajouterTache(titre) qui crée un objet tâche avec id, titre et statut terminee=false, puis l'ajoute à la liste.
delete-task.js	Importe la liste depuis add-task.js. Fournit la fonction supprimerTache(id) qui recherche une tâche par son identifiant et la supprime de la liste avec splice().
complete-task.js	Importe la liste depuis add-task.js. Fournit la fonction marquerTerminee(id) qui recherche une tâche par son id et change son attribut terminee à true.
index.js	Point d'entrée principal. Importe et teste les 3 modules en séquence : ajout de 3 tâches, marquage de la tâche 1 comme terminée, suppression de la tâche 2, affichage de la liste finale.

2. Résultat de l'exécution :

L'application a été testée avec la commande `node index.js`. Les trois fonctionnalités fonctionnent correctement comme en témoigne la capture suivante :

Exécution de `node index.js` - Les 3 fonctionnalités testées avec succès

```
mon pc@DESKTOP-7SSGS3L MINGW64 ~/todo-list-app (main)
$ node index.js
=== Application To-Do List ===

☑ Tâche ajoutée : "Faire les courses"
☑ Tâche ajoutée : "Réviser le cours Git"
☑ Tâche ajoutée : "Rendre le TP Notion"

--- Liste des tâches ---
[
  { id: 1, titre: 'Faire les courses', terminée: false },
  { id: 2, titre: 'Réviser le cours Git', terminée: false },
  { id: 3, titre: 'Rendre le TP Notion', terminée: false }
]

--- Marquer tâche 1 comme terminée ---
☑ Tâche marquée comme terminée : "Faire les courses"

--- Supprimer tâche 2 ---
✖ Tâche supprimée : "Réviser le cours Git"

--- Liste finale ---
[
  { id: 1, titre: 'Faire les courses', terminée: true },
  { id: 3, titre: 'Rendre le TP Notion', terminée: false }
]
```

VIII. Conclusion :

Ce TP pratique a permis de maîtriser l'intégration de deux outils essentiels dans le développement logiciel professionnel moderne : Notion pour la gestion de projet et Git/GitHub pour le contrôle de version du code source.

La synchronisation entre le tableau Kanban de Notion et les branches Git constitue le point clé de ce TP. Cette approche reproduit fidèlement le workflow utilisé dans les équipes agiles en entreprise, où chaque fonctionnalité est tracée à la fois dans un outil de gestion de projet et dans le système de contrôle de version.

Voici un récapitulatif des compétences acquises :

Étape	Compétence acquise
Étape 1	Configurer un espace de travail Notion avec tableau Kanban et documentation.
Étape 2	Initialiser un dépôt Git avec branches main et develop, connecté à GitHub.

Étape 3	Synchroniser les tâches Notion avec les branches Git feature/*.
Étape 4	Documenter les problèmes rencontrés et leurs solutions dans Notion.
Étape 5	Rédiger un compte-rendu de réunion récapitulatif dans Notion.
Code	Développer une application Node.js modulaire avec 3 fonctionnalités testées.